

Farkas Bence HWO3YF

Feladat: Valós idejű kvíz rendszer

Készítsünk egy online, valós idejű kvíz rendszert - egy Kahoot jellegű alkalmazást. A kliens-szerver architektúrájú webalkalmazáson keresztül a rendszer felhasználói által létrehozott kvizekben a résztvevők kérdésekre válaszolhatnak meghatározott időkereten belül. A kvizek lebonyolítása központilag vezérelt, a kérdések megjelenése és az eredmények frissítése valós időben történik. A rendszernek biztosítania kell, hogy egy kvízben egyszerre tetszőleges számú résztvevő vehessen részt, a kérdések sorrendje, időzítése és lezárása pedig a szerver által kontrollált módon történjen.

Projekt megvalósítja:

- REST alapfeladatot (2 pont)
- SignalR alapfeladatot (1 pont)
- OAuth2 alapú autentikációt (0,5 pont)
- Nagy mennyiségű adat kezelését (0,5 pont)
- Élő chat (1 pont)

Funkcionális elemzés:

A rendszer kétféle felhasználói szerepkört különböztet meg: vendég felhasználókat és a hitelesített felhasználókat.

Vendég felhasználó számára a rendszer lehetővé teszi:

- felhasználói fiók létrehozását (felhasználónév, e-mail cím és jelszó megadásával)
- bejelentkezést (e-mail cím és jelszó megadásával)
- kvízhez való csatlakozást (becenév és PIN kód megadásával)
- az aktuális kérdés helyes válaszána lekérését, amennyiben a felhasználó csatlakozott az adott kvízhez, a kvíz már elindult, valamint az aktuális kérdés lezárásra került
- képes élő chatelni az aktuális kvíz résztvevőivel

Hitelesített felhasználó számára a rendszer biztosítja:

- kijelentkezés lehetőségét
- saját adatainak lekérdezését (például az általa létrehozott kvizek listáját)
- új kvíz létrehozását
- kvíz lebonyolítását, amely magában foglalja:
 - o a kvíz publikálását
 - o a kvíz elindítását

Modern webalkalmazások fejlesztése .NET környezetben

- az aktuális kérdés lezárását
- a következő kérdésre történő léptetést

A rendszer által alkalmazott adatvalidációs szabályok:

Felhasználó:

- felhasználónév/becenév: regisztrációnál, kvízhez csatlakozás során mindig egyedi és legalább 3 és legfeljebb 255 hosszú
- email: email formátum
- jelszó: legalább 6 hosszú, legalább 1 nagybetű, legalább 1 szám, legalább 1 speciális karakter és legalább 1 kisbetű

Kvíz:

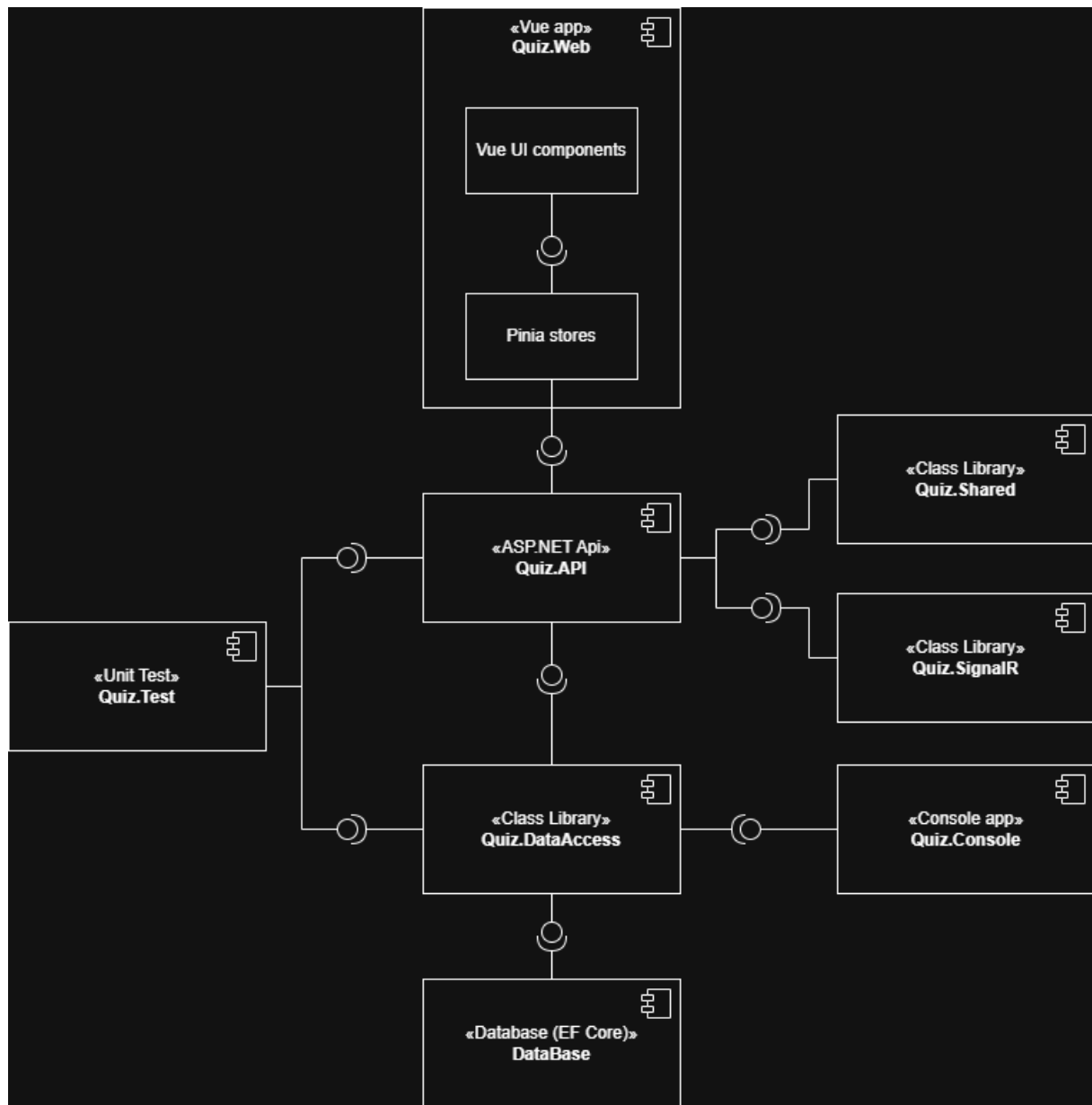
- PIN: pontosan 6 darab számjegy
- Kvíz címe: legalább 3, legfeljebb 255 hosszú
- Kérdések száma: legalább 1, legfeljebb 20

Kérdések:

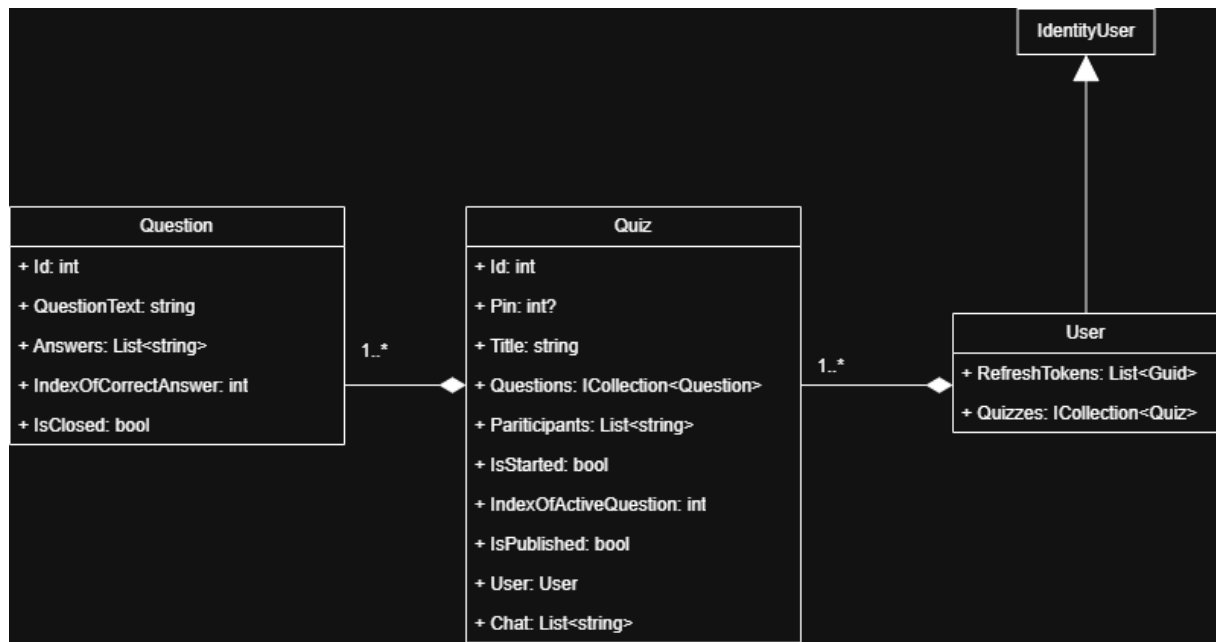
- Maga a kérdés: legalább 3, legfeljebb 255 hosszú
- Válaszok száma: legalább 2, legfeljebb 4
- Helyes választ meg kell adni a kérdéshez

(Megj. mindegyik kötelező)

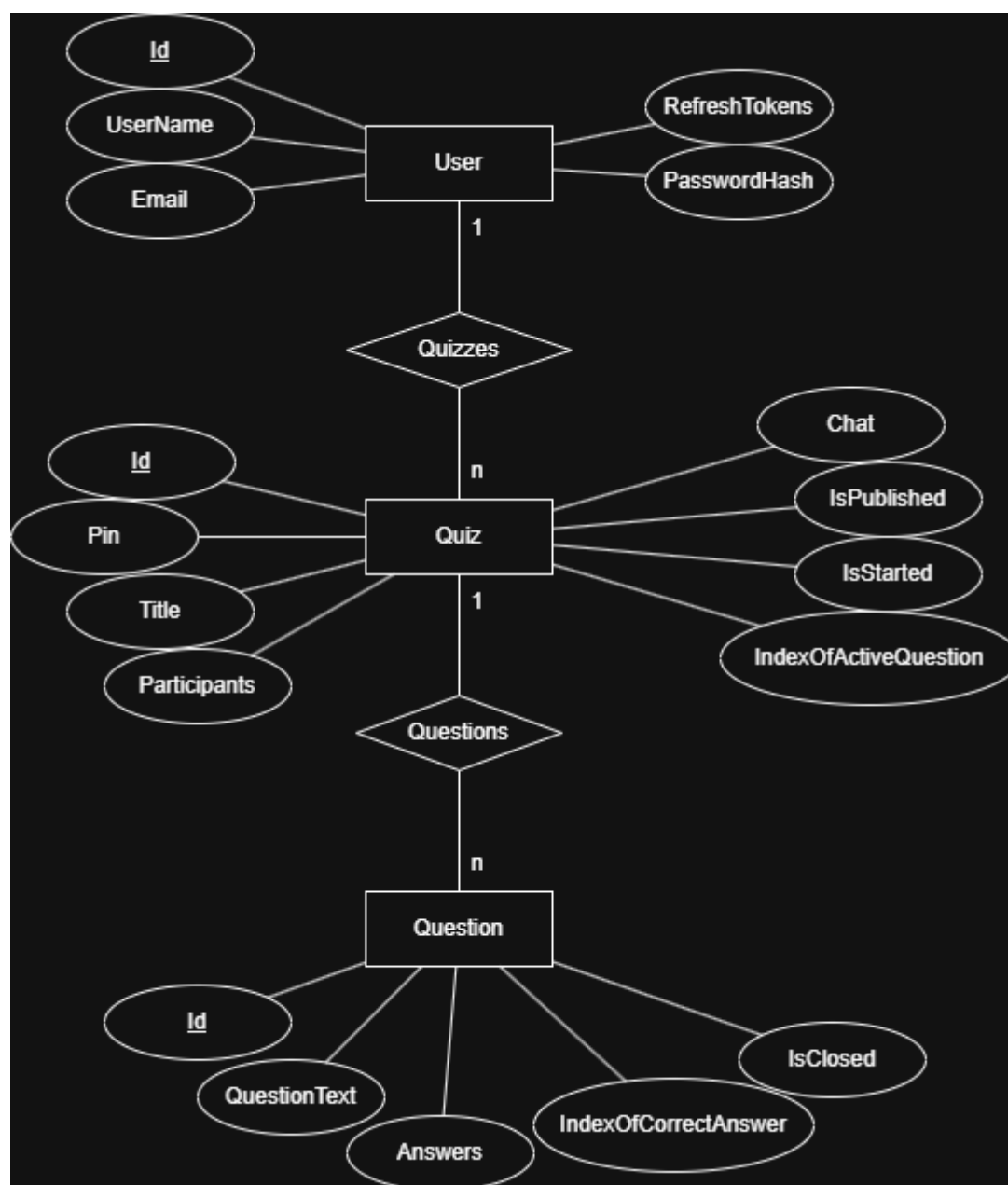
Komponens diagram:



Osztály diagram



ER diagram



Webszolgáltatás

A szürke színnel jelölt cellák a hitelesítést jelzik.

Felhasználó: /users

Metódus	Végpont	Leírás
POST	/register	Regisztráció
POST	/login	Bejelentkezés
POST	/logout	Kijelentkezés
POST	/refresh	Új access és refresh token kérése
GET	/userId	Bejelentkezett felhasználó saját adatainak lekérése

Kvíz: /quizzes

Metódus	Végpont	Leírás
GET		Publikált kvízek lekérése
POST	/quizId	Az aktuálisan belépett kvíz adatainak lekérése
POST	/create	Kvíz létrehozása
GET	/quizId/publish	Kvíz publikálása
POST	/quizId/join	Kvízhez csatlakozás
GET	/quizId/start	Kvíz elindítása
POST	/quizId/activeQuestion	Kvíz aktuális kérdésének lekérése
GET	/quizId/nextQuestion	Következő kérdésre léptetés
GET	/quizId/closeQuestion	Kérdés lezárása
POST	/quizId/activeQuestion/answer	Helyes válasz lekérése
POST	/quizId/chat	Üzenet küldés az aktuális kvíz élő chatjébe
GET	/my-quizzes/{quizId}	Authentikált felhasználó saját kvízeinek lekérése

Tesztek

Unit Tesztek

Ezek a tesztek in memory database segítségével és kizárólag mockolt függőségekkel ellenőrzi szolgáltatások összes kimenetelét

- QuizzesServiceTests
- UsersServiceTests

Controller Tesztek

Ezek a tesztek in memory database segítségével és kizárólag mockolt függőségekkel ellenőrzi a kontrollerek helyes működését

- QuizzesControllerTests
- UsersControllerTests

Integrációs Tesztek

Ezek a tesztek in memory database segítségével és valós függőségekkel, teszt környezetben ellenőrzi a komponensek közötti helyes működést

- UsersIntegrationTests
- QuizzesIntegrationTests

A legfontosabb tesztesek (UsersServiceTests)

RegisterAsync_ThrowsInvalidDataException_WhenEmailsTaken:

Ellenőrzi, hogy már foglalt e-mail címmel nem lehet regisztrálni.

RegisterAsync:

Ellenőrzi, hogy új e-mail címmel sikeresen létrejön a felhasználó.

LoginAsync_ThrowsAccessViolationException_WhenEmailNotFound:

Ellenőrzi, hogy nem létező e-mail címmel nem lehet bejelentkezni.

LoginAsync_ThrowsAccessViolationException_WhenPasswordIsIncorrect:

Ellenőrzi, hogy hibás jelszó esetén nem lehet bejelentkezni.

LoginAsync:

Ellenőrzi, hogy sikeres bejelentkezéskor létrejön az auth token, a refresh token, és a megfelelő user azonosító kerül visszaellenőrzésre.

GetCurrentUserAsync_ReturnsUser_WhenUserIsLoggedIn:

ellenőrzi, hogy bejelentkezés nélkül nincs aktuális felhasználó, bejelentkezett állapotban pedig a megfelelő user azonosítható.

LogoutAsync_ThrowsAccessViolationException_WhenRefreshTokenIsInvalid:

Ellenőrzi, hogy érvénytelen refresh tokennel nem lehet kijelentkezni.

LogoutAsync:

Ellenőrzi, hogy sikeres kijelentkezéskor megtörténik a kijelentkeztetés, és a refresh token törlődik.

RedeemRefreshTokenAsync_ThrowsAccessViolationException_WhenRefreshTokenIsInvalid:

Ellenőrzi, hogy hibás formátumú refresh token esetén hibát dob.

RedeemRefreshTokenAsync_ThrowsAccessViolationException_WhenRefreshTokenIsNotFound:

Ellenőrzi, hogy nem létező refresh token esetén hibát dob.

RedeemRefreshTokenAsync:

Ellenőrzi, hogy érvényes refresh token esetén új tokenek jönnek létre, a régi token törlődik, és a user frissítése megtörténik.

GetUserAsync_ThrowsInvalidOperationException_WhenUserIsNotLoggedIn:

Ellenőrzi, hogy bejelentkezés nélkül nem lehet felhasználói adatokat lekérni.

GetUserAsync_ThrowsInvalidOperationException_WhenUserIsNotMatching:

Ellenőrzi, hogy másik felhasználó adatait nem lehet lekérni.

GetUserAsync:

Ellenőrzi, hogy a bejelentkezett felhasználó saját adatai és kvizei megfelelően, lapozással kerülnek ellenőrzésre.